

Sampling-Geometry Band-Limits

for Radiance Fields

NeRF band-limits the radiance field globally, by hand. 3DGS does not band-limit it at all.

The limit the cameras actually imply is local, directional, and computable.

Rickarya

23EE30019 · Signal Processing & Machine Learning

Mildenhall et al., NeRF (ECCV 2020) · Kerbl et al., 3D Gaussian Splatting (SIGGRAPH 2023) · Yu et al., Mip-Splatting (CVPR 2024)

What this talk argues

PART I Radiance fields

How NeRF posed novel-view synthesis, and why it was unusably slow.

PART II 3D Gaussian Splatting

The representation, the rasteriser, and the heuristics holding it together.

PART III The gap

Every filtering decision in 3DGS ignores the geometry of the capture.

PART IV Proposal

One limit, derived from the camera geometry, tested on both representations.

Take-away: 3DGS solved speed. It did not solve which details the data actually support.

Parts I–II extend my EE60030 course project (Geometric Methods in Computer Vision, April 2026); Parts III–IV are new.

Novel-view synthesis

- Input: a set of photographs of a static scene, plus camera poses from Structure-from-Motion (COLMAP), which also yields a sparse point cloud for free.
- Output: a representation that can render the scene from viewpoints never photographed.
- The catch: the photographs are a finite, non-uniform, irregular sampling of a 5D plenoptic function. Everything downstream is a reconstruction problem.
- Three axes are always in tension — visual quality, training time, and rendering speed.

Before 2023, no method held all three at once.

Three axes

Quality

PSNR / SSIM / LPIPS on held-out views

Training

minutes vs. days on one GPU

Rendering

≥ 30 fps at 1080p, or it is not real-time

One image-formation model, two renderers

Volumetric ray-marching (NeRF)

$$C = \sum_i T_i (1 - \exp(-\sigma_i \delta_i)) c_i$$

Point-based α -blending (splatting)

$$C = \sum_i c_i \alpha_i \prod (1 - \alpha_j)$$

These are the same equation.

- Setting $\alpha_i = 1 - \exp(-\sigma_i \delta_i)$ turns the first into the second exactly (Kerbl et al., Eqs. 1-3).
- So the difference between NeRF and 3DGS is not the physics — it is how the samples along a ray are found.
- NeRF: stochastic sampling of a continuous field. Expensive, noisy, but easy to optimise.
- Splatting: an explicit, sorted list of primitives that actually overlap the pixel. Cheap, exact, but the primitives must be created and destroyed by some other mechanism.

NeRF (2020): quality at an unusable price

- An MLP maps (position, view direction) $\rightarrow$ (density, colour). The scene lives in the network weights.
- Rendering marches rays and queries the MLP hundreds of times per pixel.
- Positional encoding and hierarchical importance sampling made it work; the MLP made it slow.
- Mip-NeRF 360 (2022) took the quality crown — and needed up to 48 hours of training and roughly 10 s per rendered frame.

48 h

Mip-NeRF 360 training time per scene. Rendering runs at 0.06 fps — about 17 s per frame. Quality was never the problem.

“We believe the benefit of increasing L is limited once 2^L exceeds the maximum frequency present in the sampled input images.” — NeRF §6.4

NeRF has a band-limit, its authors know it is a sampling limit, and they set it as one global constant. Remember this sentence — the whole proposal grows from it.

Making it fast: put the field in a data structure

Plenoxels (2022)

A sparse voxel grid storing density and spherical-harmonic colour, trilinearly interpolated. No neural network at all — proof that the MLP was never the essential part.

Instant-NGP (2022)

A multi-resolution hash encoding feeding a tiny MLP, with an occupancy grid to skip empty space. Training in minutes instead of days.

What they bought, and what they paid

- Training fell to 5–8 minutes, but rendering stayed at 3–17 fps depending on the dataset — interactive, not real-time.
- Quality plateaued 2.1 dB (Instant-NGP Big) and 4.6 dB (Plenoxels) below Mip-NeRF 360 on its own dataset; the grid structure caps achievable detail.
- Both still ray-march: many samples per pixel, most of them in empty space.

PART II

3D Gaussian Splatting

Kerbl, Kopanas, Leimkühler & Drettakis · SIGGRAPH 2023

The first method to hold quality, training speed and real-time rendering simultaneously.

The primitive

$$G(x) = \exp\left(-\frac{1}{2} (x-\mu)^T \Sigma^{-1} (x-\mu)\right)$$

$\mu \in \mathbb{R}^3$	position (mean)	3 floats
$s \in \mathbb{R}^3, q \in \mathbb{R}^4$	scale vector and rotation quaternion	7 floats
$\alpha \in \mathbb{R}$	opacity, through a sigmoid	1 float
SH, degree 3	view-dependent colour · 16×3	48 floats

59 floats total · 48 are colour · 45 of those are non-DC SH = 76 % of the model

Typical converged scene: 1–5 million Gaussians.

Why this primitive?

- Differentiable and volumetric — inherits NeRF's optimisation behaviour.
- Explicit and unstructured — can be created, moved and destroyed during optimisation.
- Projects to a 2D ellipse in closed form, so it rasterises rather than ray-marches.
- Anisotropic: one flat Gaussian can represent a whole wall.

Why Σ is not optimised directly

- A covariance matrix is meaningful only if it is positive semi-definite. Gradient descent on the six free entries of Σ will leave that set within a few steps.
- So Σ is factored the way an ellipsoid is described — a rotation and three axis lengths:

$$\Sigma = R S S^T R^T \quad \text{with } S = \text{diag}(s), \quad R = R(q), \quad q \text{ normalised to unit length}$$

- s and q are stored and optimised separately; the product is PSD by construction.
- Gradients for every parameter are derived by hand rather than left to autodiff — the paper devotes Appendix A to $\partial \Sigma' / \partial s$ and $\partial \Sigma' / \partial q$, purely to avoid autodiff overhead in the inner loop.
- This is the first place a design choice is made for speed rather than for correctness. It will not be the last.

Projection: EWA splatting

- To rasterise, the 3D Gaussian must become a 2D Gaussian in image space. Zwicker et al. (2001) gave the transform:

$$\Sigma' = J W \Sigma W^T J^T \quad W = \text{viewing transform}, \quad J = \text{Jacobian of the projective map}$$

- J is the local affine approximation to a nonlinear projection — accurate near the optical axis, degrading toward the periphery.
- Dropping the third row and column of Σ' gives the 2×2 screen-space covariance, and the splat is an ellipse.
- This same J is the object my proposed method reuses. It is already computed, every frame, for every Gaussian.

EWA splatting is a resampling filter from the point-rendering literature. 3DGS inherits its geometry but not its filtering discipline — that omission is the subject of Part III.

Optimisation and adaptive density control

$$L = (1 - \lambda) L_1 + \lambda L_{D-SSIM} \quad \lambda = 0.2$$

- SGD from the SfM point cloud. Sigmoid on α , exponential on scale.
- Every 100 iterations: prune Gaussians with $\alpha < \epsilon$, then densify.

Densification — two cases, one trigger

Both fire when the average view-space positional gradient exceeds $\tau_{\text{pos}} = 0.0002$.

Small Gaussian (under-reconstruction) → CLONE.

Large Gaussian (over-reconstruction) → SPLIT, scale divided by $\varphi = 1.6$.

Every number on this slide is a hand-tuned constant

- $\tau_{\text{pos}} = 0.0002$ • $\varphi = 1.6$ • α reset to ~ 0 every 3000 iterations • densify every 100 • SH band added every 1000 • resolution warm-up at iterations 250 and 500.
- None of these is derived from the data. Nothing connects how fine a Gaussian may become to how well the cameras actually resolve that region.
- The authors are explicit that the α -reset exists to control floaters near the input cameras — a symptom, treated with a schedule.

The tile rasteriser — where the speed comes from

1

Cull

Keep Gaussians whose 99 % confidence ellipsoid meets the frustum; a guard band rejects unstable near-plane cases.

2

Tile

Split the screen into 16×16 tiles. Instantiate each Gaussian once per tile it overlaps.

3

Sort

One 64-bit key per instance: high bits = tile ID, low 32 = quantised depth. A single global radix sort orders every splat for the whole frame.

4

Blend

One thread block per tile, front-to-back, cooperative loads into shared memory. A thread stops when its pixel saturates; the tile stops when all pixels do.

The backward pass re-traverses the same sorted lists back-to-front and recovers intermediate opacities by dividing the stored final α — no per-pixel list in global memory, and no cap on how many splats receive gradients.

Results on Mip-NeRF 360

Method	SSIM ↑	PSNR ↑	LPIPS ↓	Train	FPS	Memory
Plenoxels	0.626	23.08	0.463	25m49s	6.79	2.1 GB
Instant-NGP (Big)	0.699	25.59	0.331	7m30s	9.43	48 MB
Mip-NeRF 360	0.792	27.69	0.237	48 h	0.06	8.6 MB
3DGS — 7K iters	0.770	25.60	0.279	6m25s	160	523 MB
3DGS — 30K iters	0.815	27.21	0.214	41m33s	134	734 MB

$\approx 70\times$ faster training and $\approx 2200\times$ faster rendering than the quality leader, at parity on SSIM and LPIPS.

- The price is memory: hundreds of megabytes against Instant-NGP's tens. This is what the entire 3DGS compression literature exists to attack.
- Peak training memory can exceed 20 GB on large scenes — the practical constraint on any student project in this area. Mip-NeRF 360 was trained on 4xA100 for 12 h; 48 h is the single-GPU equivalent.

What the ablations actually show

Limited backward pass

Capping gradients at the 10 front-most splats (as Pulsar does) cost 11 dB PSNR on Truck. Unbounded depth complexity is not an optimisation — it is load-bearing.

Isotropic covariance

Forcing one radius per Gaussian visibly destroys thin structures. Anisotropy is the reason the representation is compact.

Random initialisation

Works, but degrades in the background and leaves floaters that optimisation cannot remove. SfM points matter most where views are sparse.

No split / no clone

Without split, backgrounds fail. Without clone, thin structures converge slowly. Two different failure modes, one shared trigger.

Read together: the parts of 3DGS that were derived hold up. The parts that were scheduled are where the failures live.

Limitations the authors state themselves

Artifacts in under-observed regions

Elongated, splotchy Gaussians where coverage is poor.

Popping

Large Gaussians suddenly switching blend order; the guard-band rejection is named as one cause.

No regularisation

"We currently do not apply any regularization to our optimization; doing so would help with both the unseen region and popping artifacts."

Memory

Hundreds of MB per scene; >20 GB peak during training.

Hyperparameter fragility

Position learning rate must be reduced by hand for very large scenes.

Three of these five are symptoms of one cause: nothing tells the model how much detail the data support.

PART III

Three years of follow-up — and one gap

Where the field went after SIGGRAPH 2023, and what nobody fixed.

What I have verified — and what I have not

Claim	Checked against	Status
Every 3DGS equation, hyperparameter and result quoted here	The paper, in full	Verified
Every NeRF equation, hyperparameter and ablation quoted here	The paper, in full	Verified
Mip-Splatting's filter equations and sampling-rate definition	The paper	Verified
The filter is one scalar per Gaussian, on all three axes	scene/gaussian_model.py	Verified
Recompute cadence (~290 times per 30 K run)	train.py	Verified
That no one else has made this filter anisotropic	— not yet checked —	OPEN
That SH degree is not already set per Gaussian from coverage	— not yet checked —	OPEN

The two open rows are the first fortnight's work. They are the only things that can invalidate the project — cheap to settle in September, ruinous to discover in March.

Mip-Splatting: the closest anyone has come

- CVPR 2024 Best Student Paper. Diagnosis: 3DGS Gaussians can shrink below the sampling limit of the training views, so changing the test-time scale produces erosion or dilation artifacts.
- Fix 1 — a 3D smoothing filter that caps each Gaussian's size by the maximal sampling rate the input views induce.
- Fix 2 — a 2D Mip filter replacing the fixed screen-space dilation, approximating a one-pixel box filter.

$\hat{v}_k = \max_n (1_n(p_k) \cdot f_n / d_n)$	maximal sampling rate over the N training views
$\Sigma_k \leftarrow \Sigma_k + (s / \hat{v}_k^2) \cdot I, \quad s = 0.2$	a SCALAR multiple of the identity
$\alpha_k \leftarrow \alpha_k \cdot \text{sqrt}(\Sigma_k / \Sigma_k + \cdot I)$	energy-preserving opacity correction

It is the right idea. It is a scalar, from one camera.

- $\hat{v}$ is a max over views, and the code takes min-depth and max-focal independently — so only the single most favourable view enters, and the configuration of every other camera is discarded.
- That is correct only if the capture is isotropic. Standard benchmarks mostly are, which is why the approximation has gone unchallenged.

The gap, from the baseline's own source

```
distance[valid] = torch.min(distance[valid], z[valid])           # min depth over ALL cameras
if focal_length < camera.focal_x: focal_length = ...           # max focal, one global scalar
filter_3D = distance / focal_length * (0.2 ** 0.5)
self.filter_3D = filter_3D[..., None]                           # shape [N, 1] – ONE number

# still in main:  #TODO consider focal length and image width
```

Isotropic

One scalar on all three axes. The true resolution cell is short across the ray and long along it.

One camera

min-depth and max-focal keep only the single best view. The configuration of the rest — which is what gives depth resolution — is thrown away.

No angular counterpart

SH degree is global. A Gaussian seen from a 5° cone gets the same 45 coefficients as one seen from a full hemisphere.

PART IV

Proposal: band-limit by sampling geometry

Compute the filter the cameras imply, instead of scheduling one.

Same field, same samples, three different answers

Method	Where the band-limit lives	How it is set	Granularity
NeRF (2020)	The positional-encoding band L	By hand, once, from “the maximum frequency present in the sampled input images”	One scalar for the whole model
3DGS (2023)	Nowhere	Not set — the authors state no regularisation is applied at all	Absent
Mip-Splatting (2024)	A variance added to the covariance	$\hat{v}_k = \max \text{ over views of } f_n / d_n$	One scalar per primitive
This project	A covariance added to the primitive, floored at the above	Accumulated sampling Fisher information of all views that see it	Per primitive, PER DIRECTION

“The maximum frequency present in the sampled input images” is not one number.

- It falls with depth, it differs across the ray from along it, and it varies over the sphere of viewing directions.
- NeRF collapses it to a global constant and gets away with it, because one tuned L beats none. 3DGS discards it and pays in floaters, popping and elongated Gaussians.
- The accumulated sampling Fisher information is the local, directional generalisation of the quantity NeRF's authors named in 2020.

The band-limit is anisotropic

Lateral

Set by the closest, sharpest camera. Roughly z / f per pixel.

well resolved

Axial (along ray)

A single view carries no depth information at all. Resolution comes only from parallax between views.

often poorly resolved

Angular

Set by the spread of viewing directions. Determines how many SH bands are identifiable.

wildly uneven

A scalar cannot express three different limits. Mip-Splatting borrows the lateral number and applies it axially, where it has no basis.

NeRF's own ablation already shows this empirically: encoding frequencies above what the input images support stops helping and starts hurting — $L = 15$ scores below $L = 10$.

B1 — the sampling Fisher information

$$\Lambda_k = \sum_n w_{nk} \cdot (J_n W_n)^T \Sigma_{\text{pix}}^{-1} (J_n W_n) = V \text{diag}(\lambda_1, \lambda_2, \lambda_3) V^T \quad \text{rank 2 per view}$$

$$\sigma_i^2 = \min(\max(f_k^2, s^2/\lambda_i) , (\beta z_{\min,k})^2) \quad f_k^2 = \text{Mip-Splatting's own floor}$$

$$\Sigma_{\text{filt}} = V \text{diag}(\sigma_1^2, \sigma_2^2, \sigma_3^2) V^T ; \quad \Sigma_k \leftarrow \Sigma_k + \Sigma_{\text{filt}}$$

$$\alpha_k \leftarrow \alpha_k \cdot \text{sqrt}(|\Sigma_k| / |\Sigma_k + \Sigma_{\text{filt}}|) \quad \text{determinant form of the same rule}$$

- $J_n W_n$ is the product 3DGS already forms for the EWA transform. Two hyperparameters: s (confidence, default 1) and β (cap as a fraction of distance to the nearest camera).
- When $s^2/\lambda_i \leq f_k^2$ in every direction, this collapses to $\Sigma_{\text{filt}} = f_k^2 I$ — Mip-Splatting, exactly. That is a unit test, not a claim.
- So the filter never smooths LESS than the baseline in any direction: lateral detail is safe by construction, and extra smoothing appears only along directions the data leave unconstrained.
- Λ depends only on poses. Same cadence as the baseline — about 290 recomputations per 30 K run, never at render time.

B2 — angular identifiability of spherical harmonics

- 76 % of a 3DGS model is non-DC spherical-harmonic coefficients, applied at degree 3 everywhere regardless of how the Gaussian was observed.
- The paper's own §7.1 admits SH is "sensitive to the lack of angular information", and patches it with a global schedule: one band every 1000 iterations, for every Gaussian alike.

```
A_k = Σ_n w_nk · y(d_nk) y(d_nk)T          angular Gram matrix, (L+1)2 × (L+1)2
ℓ_max(k) = max { ℓ : λ_min( A_k[ℓ] ) > τ }    mask everything above; never optimise it
```

- This is textbook parameter identifiability — does the design matrix have the rank to estimate this basis coefficient?
- Magnitude pruning removes coefficients that came out small. This prevents fitting coefficients the data cannot determine at all.
- Masking alone does not shrink a fixed-stride array — storage must become variable-length: a 2-bit degree tag plus a variable payload, which every 3DGS compression pipeline already does.
- Predicted result: a memory reduction at matched PSNR, and less view-dependent overfitting in sparsely observed regions.

What is predicted — and what is not

Regime	Prediction	Falsified if
A · NeRF, implicit side	The optimal encoding band L^* falls monotonically as the capture is degraded: full orbit > one-sided > low-parallax	L^* is flat, or moves the wrong way
Well-conditioned orbits (most of Mip-NeRF 360)	Parity within noise — the filter sits at its floor almost everywhere	Any systematic loss appears
Low-parallax, one-sided, grazing (stress suite)	Gain in held-out PSNR / LPIPS and depth error, from suppressing ray-aligned floaters	No difference on ≥ 3 of 4 protocols
Mixed focal length	Gain — max-focal currently under-smooths every Gaussian whose best view is low-resolution	Parity
Sparse view (3/6/9)	Gain with no learned prior; fewer views directly lower Λ	Loss against vanilla 3DGS
Memory (B2)	Reduction at matched PSNR, dominating naive SH truncation	Curve sits on or below truncation
Cost	Train time within a few percent; render fps unchanged	Any render-time regression

The method never sharpens, so training PSNR may fall slightly while held-out PSNR rises. That asymmetry is the signature of a generalisation gain, and reporting both is the cleanest evidence available.

Why this is a signal-processing problem

Sampling theory

Two filters, not one. The render-side anti-alias filter is set by the sharpest sampling rate — Mip-Splatting's 2D Mip filter, unchanged here. The representation-side limit is what the training views can support. Conflating them is the error in the scalar 3D filter.

Estimation theory

Λ_k is a Fisher information matrix; Λ_k^{-1} is the Cramér–Rao bound on localising a feature there. Smoothing to the CRB is refusing to claim precision the measurement does not support.

Identifiability

M2 is rank analysis of a design matrix. Masking unidentifiable coefficients is model-order selection, not a trick.

Rate-distortion

Removing coefficients the data cannot identify is source coding at the representation level, rather than entropy-coding them afterwards.

"3D Gaussian Splatting reconstructs a signal from non-uniform samples without a matched anti-alias filter."

How it will be evaluated

Benchmarks

Mip-NeRF 360 (9), Tanks & Temples (2), Deep Blending (2), Blender synthetic (8). Baselines re-run in-house: 3DGS, Mip-Splatting, 3DGS-MCMC at matched budget.

Protocol

Single-scale train, multi-scale test at $1\times$, $\frac{1}{2}\times$, $\frac{1}{4}\times$, $\frac{1}{8}\times$ — the standard test for scale robustness.

New stress suite

Four camera-subset protocols built from the same images: low-parallax, one-sided arc, mixed focal length, grazing incidence. The regime the method is predicted to win, and a contribution in itself.

Rigour

Three seeds, mean $\pm$ std, paired per-scene deltas. Seven ablations from isotropic-vs-anisotropic through clamp sensitivity. Every table regenerable by one command.

Parity on standard benchmarks is the prediction, not the risk — the floor guarantees it. The diagnostic reports what fraction of Gaussians sit above the floor, which makes parity interpretable.

Two semesters

Weeks	Work	Exit criterion
Autumn W1–2	Reproduce 3DGS and Mip-Splatting; build the evaluation harness	Within 0.2 dB of published numbers
Autumn W3–5	Derive B1; build the stress suite; implement Λ accumulation	Reduction-to-Mip-Splatting unit test passes
Autumn W6–8	First full evaluation; ablations; mid-term report and seminar	An honest table, including losses
Autumn W9–13	B2 identifiability, memory study, 13-scene run, Part-I viva	Complete main table, no missing cells
Spring S1–4	Three-seed rerun; sparse-view experiments; B3 densification	Claims supported or explicitly withdrawn
Spring S5–8	Paper draft, submission, code release, reviewer-proofing	Submission ID; public repo reproducing Table 1
Spring S9–14	Thesis, final slides, live demo, viva	Submitted

B1 alone is a complete project. B2 and B3 are upside, and are the first things to cut.

IN ONE SENTENCE

The size limit on a splatting primitive should be the inverse of the accumulated sampling Fisher information, floored at the per-view Nyquist limit — a matrix, not a scalar.

One quantity, computed from the camera poses alone, that addresses ray-aligned floaters and spherical-harmonic overfitting together.

Verifiable gap

Confirmed in the baseline's own source, not inferred from prose.

Feasible

One GPU. No pretraining. Closed-form, no added training cost.

Defensible

Nyquist, Cramér–Rao, identifiability — answerable from first principles.

References

- Kerbl, Kopanas, Leimkühler, Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM TOG 42(4), 2023.
- Chai, Tong, Chan, Shum. Plenoptic Sampling. SIGGRAPH, 2000.
- Durand, Holzschuch, Soler, Chan, Sillion. A Frequency Analysis of Light Transport. SIGGRAPH, 2005.
- Zwicker, Pfister, van Baar, Gross. EWA Volume Splatting. IEEE VIS, 2001.
- Mildenhall et al. NeRF. ECCV, 2020.
- Barron et al. Mip-NeRF 360. CVPR, 2022.
- Müller et al. Instant Neural Graphics Primitives. ACM TOG 41(4), 2022.
- Fridovich-Keil and Yu et al. Plenoxels. CVPR, 2022.
- Yu, Chen et al. Mip-Splatting: Alias-free 3D Gaussian Splatting. CVPR, 2024.
- Liang et al. Analytic-Splatting. ECCV, 2024.
- Yan et al. Multi-Scale 3D Gaussian Splatting. CVPR, 2024.
- Steiner et al. AAA-Gaussians. ICCV, 2025.
- Kheradmand et al. 3D Gaussian Splatting as Markov Chain Monte Carlo. NeurIPS, 2024.
- Jiang et al. FisherRF. ECCV, 2024. • Hanson et al. PUP 3D-GS. CVPR, 2025.
- Lassner, Zollhöfer. Pulsar. CVPR, 2021. • Bagdasarian et al. 3DGS.zip survey. CGF, 2025.

Literature surveyed 30 August 2026. Re-check before submission.